Shortest Route in a Metro Network using BFS

Algorithm / Procedure (Breadth-First Search)

1. Represent the metro network as a graph, where each station is a node and each direct connection between stations is an edge (stored as an adjacency list/dictionary).

2. Initialize a queue with a list containing only the starting station as the first path, and an empty visited set to track explored stations.

3. While the queue is not empty, remove the path at the front of the queue (FIFO order) and identify its last station.

4. If the last station equals the goal station, return this path as the shortest route (BFS guarantees the first path found to the goal uses the fewest edges).

5. Otherwise, if the station has not been visited, mark it as visited.

6. For every neighboring station connected to the current station, create a new path by extending the current path with that neighbor, and add this new path to the back of the queue.

7. Repeat steps 3-6 until the goal is found or the queue becomes empty (meaning no route exists).

8. Print the resulting path, joining station names with an arrow (->) to visually represent the route.

Why BFS works

BFS explores the graph level by level (all paths of length 1, then all paths of length 2, and so on). Because it always expands the shortest known paths first, the very first time it reaches the goal station, that path is guaranteed to have the minimum number of stops (edges) — making BFS ideal for unweighted shortest-path problems like this metro network.

Python Code

```
from collections import deque

metro = {
    "Central": ["Park", "Museum"],
    "Park": ["Central", "City Mall", "Airport"],
    "Museum": ["Central", "Library"],
    "City Mall": ["Park"],
    "Library": ["Museum", "Airport"],
    "Airport": ["Park", "Library"]
}

def bfs_shortest_path(graph, start, goal):
    queue = deque([[start]])
    visited = set()
    while queue:
        path = queue.popleft()
        station = path[-1]
        if station == goal:
            return path
        if station not in visited:
            visited.add(station)
            for neighbor in graph[station]:
```

```
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)
    return None

start = "Central"
goal = "Airport"
path = bfs_shortest_path(metro, start, goal)
print("Shortest Route:")
print(" -> ".join(path))
```

Program Output

```
Shortest Route:
Central -> Park -> Airport
```

Pytutor Link

For additional reference material, see: https://tinyurl.com/25ad180BFS

Maze Treasure Hunt using DFS

Algorithm / Procedure (Depth-First Search)

1. Represent the maze as a graph, where each room is a node and each connection to an adjacent room is a directed edge (stored as an adjacency list/dictionary).

2. Maintain a visited list to keep track of rooms that have already been explored, preventing infinite loops or repeated work.

3. Start the search at the entrance room by calling a recursive dfs() function on it.

4. Within dfs(room): if the room has not been visited, print that it is being explored and add it to the visited list.

5. Check if the current room is the goal (Treasure Room). If so, print a success message and return True to signal the search is complete.

6. Otherwise, recursively call dfs() on each neighboring room listed for the current room, going as deep as possible along one path before backtracking.

7. If a recursive call returns True (treasure found down that branch), immediately propagate True back up through all calling frames, stopping further exploration.

8. If a room has no unvisited path leading to the treasure, the function returns False, causing the search to backtrack and try the next neighbor at the previous level.

9. The search ends either when the treasure is found (True is returned all the way up) or every reachable room has been visited without success (False is returned).

Why DFS works here

DFS explores as far as possible along each branch before backtracking, using the call stack (via recursion) to remember where to return to. This makes it well suited for maze traversal, since the robot commits to a path, follows it to a dead end or the goal, and only backtracks when a branch is exhausted. Unlike BFS, DFS does not guarantee the shortest path — it only guarantees that a path will be found if one exists.

Known Limitations

• **Not shortest-path optimal:** DFS finds a path to the treasure, not necessarily the shortest one. In this maze it works out (Entrance to Treasure Room via Hallway/Kitchen/Dining Room), but that is because the graph happens to be a simple branching structure with no shorter alternative route.

• **Recursion depth risk:** For very large or deeply nested mazes, the recursive dfs() function could hit Python's recursion limit and raise a RecursionError. An iterative version using an explicit stack would be more robust for large graphs.

• **No cycle protection beyond visited list:** The visited list prevents revisiting rooms, but if the maze contained cycles and the treasure were unreachable, DFS would still visit every reachable room before returning False — this can be slow for very large mazes.

• **Order-dependent results:** Because DFS explores neighbors in the order they appear in the adjacency list, the specific path found (when multiple paths exist) depends on how the maze dictionary lists each room's neighbors.

• **Mutable global state:** The visited list is a module-level variable rather than being passed explicitly or reset between calls, which could cause bugs if dfs() were reused for multiple independent searches without clearing it first.

Python Code (Python 3.11)

```
maze = {
    "Entrance": ["Hallway", "Storage"],
    "Hallway": ["Kitchen", "Living Room"],
    "Kitchen": ["Dining Room"],
    "Dining Room": ["Treasure Room"],
    "Living Room": ["Bedroom"],
    "Bedroom": ["Bathroom"],
    "Bathroom": [],
    "Storage": ["Garage"],
    "Garage": ["Workshop"],
    "Workshop": [],
    "Treasure Room": []
}

visited = []

def dfs(room):
    if room not in visited:
        print("Exploring:", room)
        visited.append(room)
        if room == "Treasure Room":
            print("\nTreasure Found!")
            return True
        for next_room in maze[room]:
            if dfs(next_room):
                return True
    return False

print("Robot starts searching...\n")
dfs("Entrance")
```

Program Output

```
Robot starts searching...

Exploring: Entrance
Exploring: Hallway
Exploring: Kitchen
Exploring: Dining Room
Exploring: Treasure Room

Treasure Found!
```

Pytutor Link

Pytutor Link: https://tinyurl.com/25ad180DFS